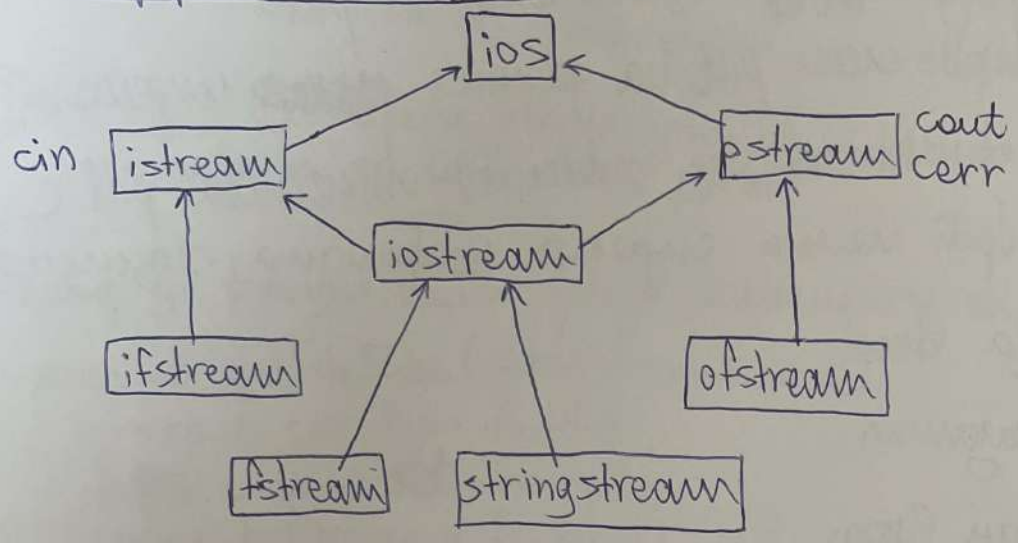
Тема 2

• поток – последователност от байтове, "насочени" в определена посока

→ Йерархия на потоците



• потоци за изход

→ имат put указател, който показва на коя позиция потокът ще "запише" ~~поставя~~ символи

→ не записват директно в посочения изход, а отварят безразмерен масив (буфер), в който записват данните временно, докато бъдат синхронизирани с изхода

форматиран изход: os << <обект>

↓

връща os и поставя обекта символ по символ

неформатиран изход:

.put(char)

.write(const char*, size) → за двоични файлове

синхронизация:

flush()-прехвърля данните от буфера във файла

позициониране:

tellp()-връща ~~[illegible]~~ позицията на put

seekp(index) - слага put на дадения ~~[illegible]~~ индекс

seekp(offset, std::ios::beg/end/cur)- отмества put с offset на брой места спрямо избраната позиция

• потоци за вход

→ get указател

форматиран вход:

ifs >> <обект>

3	7	9		A	B

→ краен масив!

int n;

ifs>>n; → чете 379

неформатиран вход:

get() - връща 1 число

get(char*, size)- void

get(char*, size, sep) → eof() по default → НЕ прескача sep

getline(char*, size, sep)

↳ '\n' по default → прескача sep

peek()-като get() но не мести указателя

ignore()- мести указ., но не връща нищо

позициониране:

tellg() - аналог на tellp()

seekg() - аналог на seekp()

! >> , getline() и ignore() прескачат '\n'

- файл - последователности от байтове
- режими на работа на потоци:

1 → ios::in → поток, отворен за четене

2 → ios::out → поток, отворен за писане

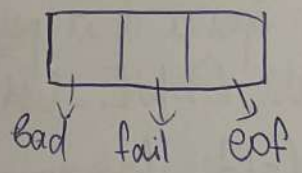
4 → ios::ate → поставя указателя накрая (и позволява местене)

8 → ios::app → поставя указателя накрая (и НЕ позволява местене)

16 → ios::trunc → изтрива съдържанието на файла (ако той съществува)

32 → ios::binary → отваря файла в двоичен формат (\n се третира различно)

- състояния на потоци:



bad() → възникнала е сериозна грешка, която оставя потока в лошо състояние и предотвратява четене/писане

fail() → I/O проблем, след който са възможни следващи операции

eof() → достигнат е край на файла

good() → и трите бита са 0

clear() → чисти състоянието на потока

Четене на поток

→ автоматично вика open()

```
std::ifstream ifs(<име на файл>, <режим>);
if(!ifs.is_open()) {...}
```

error handling → винаги проверяваме дали потокът е отворен

} → в края на scope-а автоматично се вика ifs.close()
(но е добре да го извикваме ръчно)

• Пример за функция, която връща големината на файл

```
size_t getFileSize(std::ifstream& ifs){
    size_t currPos = ifs.tellg();
    ifs.seekg(0, std::ios::end);
    size_t fileSize = ifs.tellg();
    ifs.seekg(currPos);
    return fileSize; }
```

• Пример за ф-я, която връща брой редове в текстов файл

```
const char NEW_LINE_CHAR = '\n';
int getCharCount(std::ifstream& ifs, char ch){
    if(!ifs.is_open()) { return -1; }
    int count = 0;
    while (true) {
        char current = ifs.get();
        if (ifs.eof()) { return count; }
        if(current == ch){ count++; }
    }
    return count;
}
```